MEGN 651A: Homework Four

Rachel Bertaud

March 13, 2026

The code used to produce this work can be located at this address:

https://github.com/rachbertaud/Advanced_CFD/tree/main/04_robin_bc_room_heating.

For reproduction of results, download the folder `04_robin_bc_room_heating` and enter the directory of the corresponding case you would like to reproduce. Simply run `$ make build` and `$ make run` to produce the main data files. To produce error data for all the cases besides main, enter the `Verf` directory and run `$ make build` then `$ make ErrorData`. For visualization, run `viz.m` in the respective folder to produce and save the plots provided below. For the error plots, run `errorplot.m`. Here are the following case types:

- `Verf` - Verifies inclusion of Robin conditions to Steady Advection-Diffusion solver built in HW3 (verifies main)
- `Main` - Runs the physical example we used our solver on with no sourcing
- `Heater` - Adds a heat source to main through the source term! This is our physical application.

`$ make clean` removes all the executables and all the `txt` files in the respective folders. All of the results shown are for values in table 1, except of A and the location of the space heater.

Overview of Code

The purpose of this code is to numerically solve the steady, two-dimensional advection-diffusion equation at discrete locations using a finite-volume method with physical additions to the boundaries. Our goal is to simulate the temperature distribution of air moved by a ceiling fan with a standard room with two windows, a door, and a ceiling fan.

Before we get there, we must consider the following governing equation:

$$\int_{CS} (\rho c_p \mathbf{u}\phi) \cdot \mathbf{n}\, dA = \int_{CS} k\, (\nabla\phi \cdot \mathbf{n})\, dA + \int_{CV} f\, d\mathcal{V},$$

where k is the thermal conductivity (W/m·K), ρ is the density (kg/m^3), and c_p is the specific heat capacity (J/kg·K).

This can be discretized as

$$(A_P + D_P)\phi_P = (A_E + D_E)\phi_E + (A_W + D_W)\phi_W + (A_N + D_N)\phi_N + (A_S + D_S)\phi_S + b, \tag{1}$$

where the source term is

$$b = f_P \Delta x \Delta y,$$

the diffusion coefficients are

$$D_E = \frac{\Gamma_e \Delta y}{x_E - x_P}, \quad D_W = \frac{\Gamma_w \Delta y}{x_P - x_W}, \quad D_N = \frac{\Gamma_n \Delta x}{y_N - y_P}, \quad D_S = \frac{\Gamma_s \Delta x}{y_P - y_S},$$

$$D_P = D_E + D_W + D_N + D_S,$$

and the advection coefficients are

$$A_E = -F_e\sigma_e\Delta y, \quad A_W = F_w(1-\sigma_w)\Delta y, \quad A_N = -F_n\sigma_n\Delta x, \quad A_S = F_s(1-\sigma_s)\Delta x,$$

$$A_P = F_e(1-\sigma_e)\Delta y - F_w\sigma_w\Delta y + F_n(1-\sigma_n)\Delta x - F_s\sigma_s\Delta x.$$

We further define the following:

$$\sigma_e = \frac{x_e - x_P}{x_E - x_P}, \quad \sigma_w = \frac{x_w - x_W}{x_P - x_W}, \quad \sigma_n = \frac{y_n - y_P}{y_N - y_P}, \quad \sigma_s = \frac{y_s - y_S}{y_P - y_S}.$$

Note that we will analytically solve for $c(x, y)$ using linear algebra. We solve

$$\mathbf{Ac} = \mathbf{x}$$

for $\mathbf{c}$ given $\mathbf{x}$ and $\mathbf{A}$, which are built using the discretized equations above.

Robin Boundary Conditions

The novel addition to this assignment was implementing Robin conditions to add windows and doors to our boundaries. The following boundary conditions were applied:

$$\begin{aligned}
T\Big|_{y=0} &= T_d, \qquad s_1 \le x \le s_2, \\
-k\frac{\partial T}{\partial n}\Big|_{x=0} &= h\left(T\Big|_{x=0} - T_{\text{ext}}\right), \qquad w_1 \le y \le w_2, \\
-k\frac{\partial T}{\partial n}\Big|_{y=L_y} &= h\left(T\Big|_{y=L_y} - T_{\text{ext}}\right), \qquad n_1 \le x \le n_2, \\
\frac{\partial T}{\partial n}\Big|_{\text{wall}} &= 0,
\end{aligned}$$

where s, w, and n defined the respective x or y values the door or windows appear on. Define T_d as the temperature of the door, measured in Kelvin and T_{ext} as the external temperature of the room. I implemented these through a new structure called `BC`, which took in values for x and y to produce values a, b, g that fit this form:

$$aT + b\frac{\partial T}{\partial n} = g.$$

The sign convention on the normal derivative is very important. These were applied in the definition of the boundary conditions in the construction of A.

Verification of Main

We first define our manufactured solution for the verification steps as

$$T_e(x, y) = \cos(x)\cos(y).$$

We discretize as defined for the steady, 2D advection-diffusion equation with boundary conditions for each section that fits the form

$$aT + b\frac{\partial T}{\partial n} = g.$$

These are quick computations and will not be defined here. We define the following forcing term for our manufactured solution:

$$f_p(x, y) = -k\nabla^2 T_e + \rho c_p\left(u\frac{\partial T_e}{\partial x} + v\frac{\partial T_e}{\partial y}\right)$$

$$u(x, y) = -\frac{A}{b_y L_y}\sin\left(\frac{2\pi x}{b_x L_x}\right)\cos\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s},$$

$$v(x,y) = \frac{A}{b_x L_x} \cos\left(\frac{2\pi x}{b_x L_x}\right) \sin\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s},$$

where u and v are the velocity components (m/s), A is the flow amplitude (m^2/s), and b_x, b_y, L_x, L_y are prescribed dimensionless constants that determine the spatial structure of the flow and dimensional length scales (m), respectively.

We then build **A** and **b** as explicitly defined in the overview section, and solve for **c**. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, **A**, computes the expected solution for the 2D steady advection diffusion equation with the new Robin boundary conditions.

The results of the verification process are defined below in the "Results" section.

Error Analysis

During the verification process, error of the numerical solution is calculated as relative error using the infinity norm, which is defined as

$$E_\infty = \frac{||c_{\text{numerical}} - c_{\text{analytical}}||_\infty}{||c_{\text{analytical}}||_\infty} = \frac{\max|c_{\text{numerical}} - c_{\text{analytical}}|}{\max|c_{\text{analytical}}|}.$$

Note that we will also define local error as

$$\text{Local Error} = |c_{\text{numerical}} - c_{\text{analytical}}|.$$

Results

Overall, the results of this code verified its capabilities and produced an intuitive solution to our little room. Everything performed as anticipated. Refer to

- Figure 1 for results of `Verf` — where we confirmed second-order on our new Robin conditions.
- Figure 2 for results of `Main` — where we investigated the cooling of the room with no heat source.
- Figure 3 for results of `Heater` — where we added a heat source and investigate its placement!

Problem 5.3 (Simulating a room with no heater)

I chose to do an average as the mean of the field. To maximize room temperature, the winner from the mean calculation is counterclockwise flow. This intuitively makes sense, as the heat is being pushed around the room towards the most insulated corner — which helps spread the heat from the door around the room. Visually, most of the room is at a "medium" temperature, and we can see the heat band travel from the door all the way to the window on the north boundary! The counterclockwise flow had a mean of 275 K. It was not too far ahead of the clockwise flow - but the addition space to travel without heat loss seems to be a key factor.

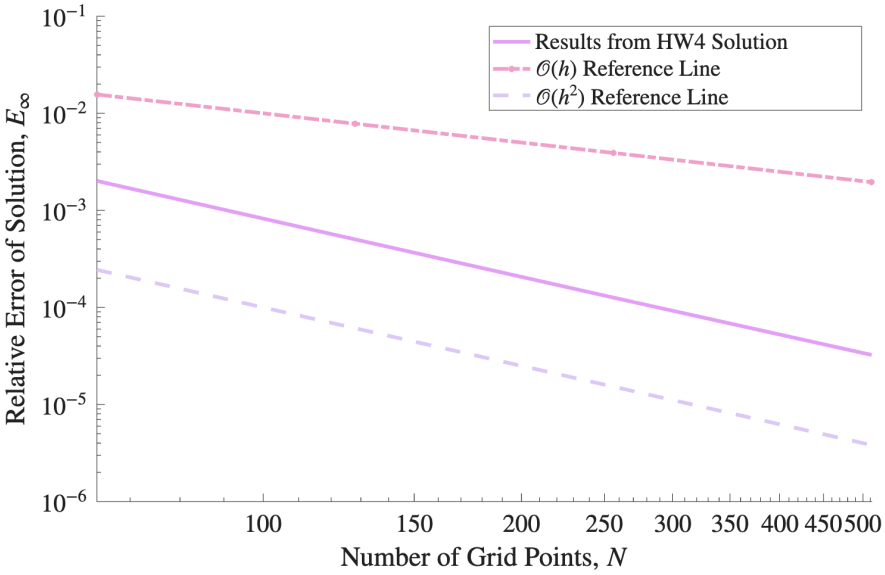


Figure 1: Relative ℓ_∞ error E_∞ of the numerical solution as a function of the number of grid points N, compared against $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ reference lines. The solution converges at second-order, consistent with the $\mathcal{O}(h^2)$ reference.

HW3_C_Numerical.png

(a) Clockwise flow. $\bar{T} = 273$ K. ($A = 0.01$).

HW3_CC_Numerical.png

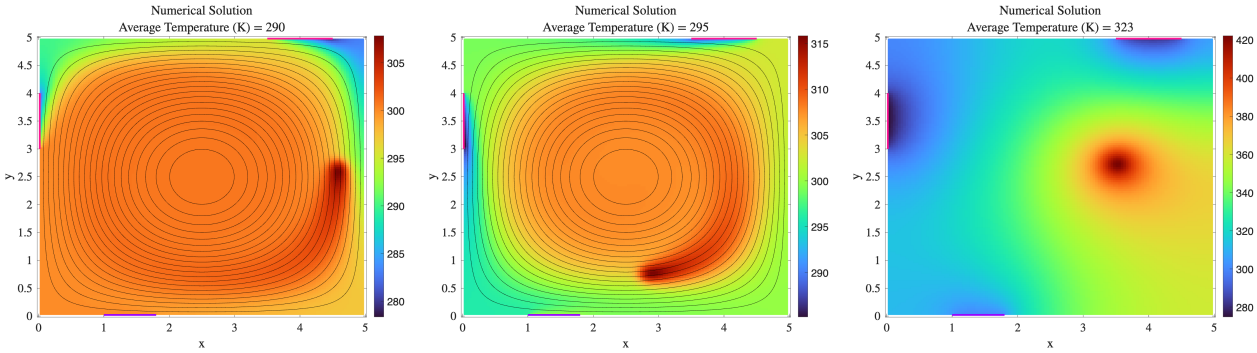
(b) Counter-clockwise flow ($A = -0.01$). $\bar{T} = 275$ K.

HW3_Stale_Numerical.png

(c) No flow.

Figure 2: Numerical temperature solutions for three flow configurations. Pink and purple boundary markers indicate windows and the door respectively.

Problem 5.4 (Simulating a room with a space heater)



(a) Clockwise flow. $\bar{T} = 290$ K. ($A = 0.01$).

(b) Counter-clockwise flow ($A = -0.01$). $\bar{T} = 295$ K.

(c) Stale flow. $\bar{T} = 323$ K.

Figure 3: Numerical temperature solutions with streamlines for three flow configurations. Pink and purple boundary markers indicate windows and the door respectively.

I had a little fun in considering what would happen if we added a source term that represented a space heater. I ended up going for a Gaussian definition:

$$f(x, y) = A \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_X^2} + \frac{(y - y_0)^2}{2\sigma_Y^2}\right)\right),$$

where

- A — amplitude, scales the peak value of the Gaussian
- x_0, y_0 — center of the Gaussian in x and y respectively
- σ_X, σ_Y — standard deviations controlling the width/spread of the Gaussian in the x and y directions.

There was in-class discussion about what value to set for Q. I went with 75 (questionable units), as typical space heater values of approximately 1500 blew the code up — a general trend for my classmates. I chose the radius to be approximately 0.2 m, which is about 8 inches for my Americans. This seemed pretty normal for a space heater.

Without doing a proper optimization process, after messing around and a little sparkle of intuition, the locations I decided upon for my space heater are shown in the figures above. Note that all of the color bars are not the same, but at least pretty similar. The winner of averages was the stale room. This is a good time to trust our visual instincts, as there is a huge gradient in one location that is skewing the mean, while a good chunk of the room is on the colder side. Visually speaking, the winner to me is clockwise. It is able to spread the heat from the source to more of the room — even making it slightly difficult to pinpoint the window on the west side of the room. It won for similar reasons as no heat source — it was able to spread most of its heat through the insulated section of the room.

Table

Parameter	Value	Description	Units
Case Inputs			
ρ	1.225	Density of air	kg/m^3
C_p	1005	Heat capacity of air	J/(kg·K)
k	0.026	Thermal conductivity	W/(m·K)
A	0.01	Velocity amplitude	m/s
b_x	2	Flow parameter	—
b_y	2	Flow parameter	—
Mesh Inputs			
N_x	128	Number of cells in x	—
N_y	128	Number of cells in y	—
L_x	5	Length of domain in x	m
L_y	5	Length of domain in y	m
Boundary Condition Inputs			
k	0.026	Thermal conductivity of air	W/(m·K)
h	0.1	Heat transfer coefficient (windows)	W/(m^2·K)
T_{ext}	260	External temperature	K
T_d	300	Door temperature	K
T_s	75	Heater temperature	Unsure :)
h_1, h_2	4.58, 2.71	Heater coordinates	m
n_1, n_2	3.5, 4.5	North window coordinates	m
w_1, w_2	3, 4	West window coordinates	m
s_1, s_2	1, 1.8	South door coordinates	m

Table 1: Simulation input parameters.

Code

For the sake of page usage, I am only including the code that builds the LHS/RHS and the main code that runs the solver. Everything else that was used to produce these results can be found at this GitHub location. Please refer to the first page for reproduction instructions.

---

main.cpp

```cpp
#include "Header_Case.h" // needed for case functions/structs
#include "Header_Mesh.h" // needed for mush functions/structs
#include "Header_matrix.h" //need for initalizing/building A and RHS
#include "Header_EigenSolve.h" // needed for solver (EigenSolve)
#include "Header_ScaSF.h" // needed for flux/phi stuff
#include "Header_Robin.h" // needed for BC

int main()
{

        caseType Case;          //declare case
        Case.read();
        Case.disp();

        meshType Mesh;
        Mesh.read();
        //std::cout << "here" << std::endl;
        Mesh.disp();
        //std::cout << "here" << std::endl;
        Mesh.create();
        //std::cout << "here" << std::endl;
        Mesh.write();
        //std::cout << "here" << std::endl;


        matrixType A;
        matrixType RHS;

        BC BC;
        BC.read(); //read BC inputs
        BC.disp(); //display BC inputs




        A.init(Mesh);
        RHS.init(Mesh);

        surfaceScalarField Flux;
        Flux.allocate(Mesh);
        Flux.ComputePhix(Mesh, Case);
        Flux.ComputePhiy(Mesh, Case);

        A.BuildA(Mesh, Case, Flux, BC);
        RHS.BuildRHS(Mesh, Case, BC);

        int flag = EigenSolve(A, RHS);

        return flag;
}
```

funcs_matrix.cpp

```cpp
#include <vector>

#include "Header_matrix.h"      // for builder funcs/structs
#include "Header_Mesh.h"        // for mesh funcs/structs
#include "Header_Case.h"        // for case funcs/structs
#include "Header_ScaSF.h"               // for advection funcs/structs
#include "Header_Robin.h"

void matrixType::init(const meshType& Mesh)
{
        Nx = Mesh.Nx;
        Ny = Mesh.Ny;
        Nx2 = Nx + 2;
        Ny2 = Ny + 2;
        B.assign(Nx2*Ny2, 0.0);
}

void matrixType::addEntry(int row, int col, double val)
{
        ROW.push_back(row);
        COL.push_back(col);
        VAL.push_back(val);
}

void matrixType::setRHS(int i, int j, double val)
{
        B[i + j*Nx2] = val;
}

double matrixType::getRHS(int i, int j) const
{
        return B[i + j*Nx2];
}

void matrixType::BuildA(
             const meshType& Mesh,
             const caseType& Case,
             const surfaceScalarField& Flux,
             BC& BC
           )
{
        double dx = Mesh.dx;
        double dy = Mesh.dy;
        double k = Case.k;

        int row;

        //south/north boundary conditions
        for (int i = 1; i <= Nx; i++)
        {
                // south (Robin)
                row = i;
                double as,bs,gs;
                BC.S(Mesh.xc[i], 0.0, as, bs, gs);
                addEntry(row, row , 0.5*as + bs/dy);
                addEntry(row, row + Nx2, 0.5*as - bs/dy);
```

```
57
58                  //north (Robin)
59                  row = i + (Nx2*(Ny + 1));
60                  double an,bn,gn;
61                  BC.N(Mesh.xc[i], 0.0, an, bn, gn);
62                  addEntry(row, row, 0.5*an + bn/dy);
63                  addEntry(row, row - Nx2, 0.5*an - bn/dy);
64          }
65
66          //east/west boundary conditions
67          for (int j = 1; j <= Ny; j++)
68          {
69                  //west (Robin)
70                  row = Nx2*j;
71                  double aw,bw,gw;
72                  BC.W(Mesh.yc[j], 0.0, aw, bw, gw);
73                  addEntry(row, row, 0.5*aw + bw/dx);
74                  addEntry(row, row + 1, 0.5*aw - bw/dx);
75
76                  //east (Neumann (through robin))
77                  row = Nx2*(j + 1) - 1;
78                  double ae,be,ge;
79                  BC.E(Mesh.yc[j], 0.0, ae, be, ge);
80                  addEntry(row, row, ae*0.5 + be/dx);
81                  addEntry(row, row - 1,ae*0.5 - be/dx);
82          }
83
84          //five-point stencil
85          double De  = k*dy/dx;
86          double Dw = k*dy/dx;
87          double Ds = k*dx/dy;
88          double Dn = k*dx/dy;
89          double Dp = De + Dw + Ds + Dn;
90
91
92          double Se = 0.5;
93          double Sw = 0.5;
94          double Sn = 0.5;
95          double Ss = 0.5;
96
97
98          double cp = Case.cp;
99
100         //internal nodes of A
101
102         for (int i = 1; i <= Nx; i++)
103         {
104                 for (int j = 1; j <= Ny; j++)
105                 {
106
107                         double Fe = Flux.gete(i,j);
108                         double Fw = Flux.getw(i,j);
109                         double Fs = Flux.gets(i,j);
110                         double Fn = Flux.getn(i,j);
111
112                         double Ae = -Fe*Se*dy;
113                         double Aw = Fw*(1 - Sw)*dy;
114                         double As = Fs*(1 - Ss)*dx;
115                         double An = -Fn*Sn*dx;
```

```
116                         double Ap = Fe*(1 - Se)*dy - Fw*Sw*dy + Fn*(1 - Sn)*dx -
                                Fs*Ss*dx;
117
118                         double ae = De + Ae;
119                         double aw = Dw + Aw;
120                         double as = Ds + As;
121                         double an = Dn + An;
122                         double ap = Dp + Ap;
123
124                         row = Nx2*j + i;
125
126                         addEntry(row, row, ap);
127                         addEntry(row, row + 1, -ae);
128                         addEntry(row, row - 1, -aw);
129                         addEntry(row, row + Nx2, -an);
130                         addEntry(row, row - Nx2, -as);
131                 }
132         }
133
134         // extraneous nodes
135
136         //bottom left
137         addEntry(0,0,1);
138
139         //bottom right
140         addEntry(Nx2 - 1, Nx2 - 1, 1);
141
142         //top left
143         addEntry(Nx2*(Ny + 1), Nx2*(Ny + 1), 1);
144
145         //top right
146         addEntry((Nx2*Ny2 - 1), (Nx2*Ny2 - 1), 1);
147
148 }
149
150 void matrixType::BuildRHS(
151                 const meshType& Mesh,
152                 const caseType& Case,
153                   BC& BC)
154 {
155
156         double dx = Mesh.dx;
157         double dy = Mesh.dy;
158         double Lx = Mesh.Lx;
159         double Ly = Mesh.Ly;
160
161         double k = Case.k;
162         double by = Case.by;
163         double bx = Case.bx;
164
165         // south/north BC (Dirichlet)
166         for (int i = 1; i <= Nx; i++)
167         {
168                 //south
169                 // double gs = Case.Ts;
170                 double xs = Mesh.xc[i];
171                 double as = 0;
172                 double bs = 0;
173                 double gs = 0;
```

```
174                     BC.S(xs, 0.0, as, bs, gs);
175
176                     setRHS(i, 0, gs);
177
178                     //north
179                     double xn = Mesh.xc[i];
180                     double an = 0;
181                     double bn = 0;
182                     double gn = 0;
183                     BC.N(xn, 0.0, an, bn, gn);
184
185                     setRHS(i, Ny + 1, gn);
186             }
187
188             // east/west BC (Neumann)
189             for (int j = 1; j <= Ny; j++)
190             {
191                     //west
192                     double yw = Mesh.yc[j];
193                     double aw = 0;
194                     double bw = 0;
195                     double gw = 0;
196                     BC.W(yw, 0.0, aw, bw, gw);
197
198                     setRHS(0, j, gw);
199
200                     //east
201                     double ye = Mesh.yc[j];
202                     double ae = 0;
203                     double be = 0;
204                     double ge = 0;
205                     BC.E(ye, 0.0, ae, be, ge);
206
207                     setRHS((Nx + 1), j, ge);
208             }
209
210     }
```